

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**АННОТАЦИЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(БАКАЛАВРСКУЮ РАБОТУ)
ПО НАПРАВЛЕНИЮ ПОДГОТОВКИ
09.03.01 – «ИНФОРМАТИКА И ВЫЧИСЛИТЕЛЬНАЯ ТЕХНИКА»**

**НАПРАВЛЕННОСТЬ (ПРОФИЛЬ) ОБРАЗОВАТЕЛЬНОЙ ПРОГРАММЫ
«ИНФОРМАТИКА И ВЫЧИСЛИТЕЛЬНАЯ ТЕХНИКА»**

Тема

**Реализация языкового сервера для языка описания
криптографических примитивов Jasmin**

Выполнил

Бучнев Александр Андреевич

подпись

Иннополис, Innopolis, 2025

Оглавление

1	Введение	4
1.1	Язык программирования Jasmin	4
2	Обзор литературы	6
2.1	Протокол языкового сервера	6
2.2	Компилятор Jasmin	8
2.3	Tree-sitter	8
3	Методология	9
3.1	Требования и подход к разработке	9
4	Реализация	11
4.1	Архитектура языкового сервера Jasmin	11
4.2	Реализованные функции	12
4.3	Поддержка Tree-sitter	13
5	Выводы	14
	Список использованной литературы	15

Аннотация

Разработчики программного обеспечения создают языки программирования для конкретных целей, и эффективное использование этих языков требует качественной поддержки при редактировании кода. Представленный в 2016 году протокол языкового сервера (Language Server Protocol) призван помочь разработчикам получить расширенную поддержку в предпочитаемом ими текстовом редакторе (vim, neovim) или IDE (VSCode, Eclipse IDE) путём разделения «языковой» и «редакторской» частей.

Поскольку современные IDE и некоторые текстовые редакторы поддерживают протокол языкового сервера «из коробки», для получения ряда функций редактирования разработчикам достаточно реализовать всего один программный компонент — так называемый языковой сервер. По какой-то причине не для всех языков программирования создаётся такая поддержка. Одним из таких языков является Jasmin. Jasmin — это язык описания криптографических примитивов, и его основными пользователями являются исследователи в области криптографии. В данной работе я представляю реализацию языкового сервера для языка программирования Jasmin, который поддерживает диагностику, переход к определению, инкрементальный синтаксический анализ, подсветку символов, подсветку синтаксиса и отображение типа символа при наведении курсора.

Глава 1

Введение

Протокол языкового сервера (LSP), представленный в 2016 году [1], позволяет отделить логику поддержки языка от редактора кода. Это решает проблему $M \times N$, позволяя одному языковому серверу работать с множеством редакторов и IDE.

1.1 Язык программирования Jasmin

Криптографическое ПО играет ключевую роль в безопасности, и уязвимости в нём, могут иметь катастрофические последствия. Для снижения рисков создания ПО с ошибками создаются фреймворки, позволяющие формально верифицировать криптографические примитивы. Одним из таких фреймворков является Jasmin [2].

Jasmin — это фреймворк для реализации и верификации криптографических примитивов. Его компилятор позволяет транслировать код в спецификации EasyCrypt [3] для формального анализа. Данная работа посвящена реализации языкового сервера для Jasmin.

В данной работе мы представляем реализацию инфраструктуры языкового сервера для языка программирования Jasmin.

Основной вклад этой работы — реализация языкового сервера, который поддерживает:

- Диагностику для языка Jasmin
- Переход к определению (`textDocument/definition`)
- Подсветку ссылок (`textDocument/highlight`)
- Отображение типов при наведении (`textDocument/hover`)
- Поддержку Tree-sitter для языка Jasmin

Глава 2

Обзор литературы

2.1 Протокол языкового сервера

Протокол языкового сервера (LSP) [1] — это языко-независимый стандарт, который упрощает интеграцию функциональности языка программирования в различные среды разработки. Он решает проблему необходимости создавать отдельную реализацию для каждой пары «язык-редактор».

Реализации языковых серверов

Для оптимальной интеграции с компилятором `Jasmin`, написанным на `OCaml`, языковой сервер также был разработан на `OCaml`. Ниже анализируются релевантные реализации на `OCaml` для выявления лучших практик.

Языковой сервер `Jasmin`

Помимо нашей реализации, существует только один сервер для `Jasmin` [4], который предоставляет лишь базовую диагностику и обладает рядом

недостатков.

Языковой сервер OCaml (Merlin)

Merlin [5] — это продвинутый сервер для OCaml, разработанный до появления LSP [6]. Его ключевое преимущество — качественная обработка неполного или некорректного кода, что позволяет предоставлять точную диагностику и автодополнение в реальном времени. Архитектура Merlin является эталоном глубокой интеграции с IDE.

Языковой сервер Semgrep

Языковой сервер Semgrep [7] послужил основным техническим ориентиром благодаря чистому и модульному дизайну. Он использует стандартные библиотеки `ocaml-lsp` [8] для работы с протоколом и `lwt` [9] для асинхронных операций. Применение чистых функций упрощает отладку и добавление новой функциональности.

Библиотеки для разработки языковых серверов

Для разработки была использована библиотека `ocaml-lsp` [8], широко применяемая в сообществе OCaml. Она обрабатывает сообщения JSON-RPC 2.0 [10], преобразуя их в типизированные структуры OCaml с помощью библиотеки `yjson` [11]. Это позволяет разработчикам сосредоточиться на логике языка, а не на деталях протокола.

2.2 Компилятор Jasmin

Компилятор Jasmin [2] — это формально верифицированный инструмент для компиляции криптографического кода в безопасный ассемблерный код. Встроенный статический анализатор проверяет такие свойства, как безопасность памяти и завершаемость. Компилятор может экспортировать программы в спецификации EasyCrypt [3] для дальнейшей формальной верификации. Основная сложность при работе с ним — отсутствие публичного API.

2.3 Tree-sitter

Tree-sitter [12] — это генератор парсеров для построения быстрых и отказоустойчивых синтаксических анализаторов, которые создают конкретные синтаксические деревья (CST). Его ключевое преимущество — инкрементальный парсинг, который позволяет эффективно обновлять синтаксическое дерево при небольших изменениях в коде, что крайне важно для текстовых редакторов и IDE.

Глава 3

Методология

3.1 Требования и подход к разработке

В данном разделе мы подробно описываем требования к функциональности языкового сервера.

Требования

Языковой сервер должен поддерживать ключевые функции для разработки:

- Отображение информации о символе при наведении (подсказка о типе)
- Переход к определению
- Поддержка синтаксической/семантической подсветки
- Диагностика исходного кода (парсинг, проверка типов)

Также важно предоставить не только исполняемый файл сервера, но и чёткие инструкции по его интеграции с редактором.

Архитектура

Основываясь на архитектуре серверов для OCaml и Semgrep (см. 2.1), была выбрана расширяемая и асинхронная архитектура. Сервер реализован на OCaml для лёгкой интеграции с компилятором Jasmin, который написан на этом же языке.

Реализация разделена на два компонента: сервер JSON-RPC и группа обработчиков LSP. Сервер отвечает за низкоуровневые операции протокола, а обработчики реализуют логику.

Глава 4

Реализация

4.1 Архитектура языкового сервера Jasmin

В этом разделе представлена реализация языкового сервера (LSP) для Jasmin, разработанного на OCaml. Сервер напрямую интегрируется с фронтом компилятора Jasmin, используя его систему типов и абстрактное синтаксическое дерево (AST) для предоставления функциональности через протокол LSP.

Ключевые части логики реализованы в виде чистых функций, что упрощает отладку и расширение. Сервер работает как автономный исполняемый файл и управляется через конечный автомат состояний (*Uninitialized*, *Running*, *Stopped*), асинхронно обрабатывая запросы клиента в отдельных потоках `lwt`. Реализовано конфигурируемое логирование для удобства отладки.

4.2 Реализованные функции

Диагностика кода в реальном времени

Диагностика запускается при сохранении файла и позволяет выявлять синтаксические и типовые ошибки без вызова компилятора. Процесс выполняется в отдельном потоке и состоит из двух этапов: сначала парсинг кода, затем — проверка типов с помощью API компилятора. Текущая реализация не поддерживает инкрементальный анализ и обрабатывает файл целиком.

Подсветка символов и навигация

Для подсветки всех использований символа сервер находит соответствующий узел в AST и ищет все его вхождения, используя уникальные идентификаторы, которые компилятор присваивает каждому символу. Это позволяет корректно различать переменные с одинаковыми именами.

Переход к определению

Функция перехода к определению ('Go to definition') также реализована с помощью AST, поскольку узлы дерева уже содержат информацию о местоположении определения символа.

Отображение типа при наведении

Для отображения типа символа при наведении курсора используется API компилятора, а именно его функции для форматированного вывода (pretty-printing). Это позволяет получать и отображать информацию о типе напрямую из AST без дополнительных преобразований.

4.3 Поддержка Tree-sitter

Для подсветки синтаксиса был разработан парсер на основе Tree-sitter [12]. Грамматика была адаптирована из существующей грамматики для Menhir [13], используемой в компиляторе Jasmin.

Tree-sitter строит конкретное синтаксическое дерево (CST) кода и использует специальные запросы ('highlight queries') для раскраски синтаксических конструкций. В отличие от семантической подсветки, этот механизм основан только на структуре кода, но значительно улучшает читаемость и удобство работы в редакторе.

Глава 5

Выводы

Наша реализация языкового сервера превосходит существующее решение за счёт дополнительных функций, однако она требует доработки. Необходимо добавить такие функции, как контекстно-зависимое автодополнение кода и поддержка переименования. Кроме того, нужно реализовать инкрементальный парсинг, что потребует разработки собственного фрон-тенда для компилятора Jasmin для эффективной обработки небольших изменений в коде. Также будет полезна возможность форматирования кода с настраиваемыми опциями для улучшения читаемости.

Список использованной литературы

- [1] Microsoft, *Official page for Language Server Protocol*, 2016. url: <https://microsoft.github.io/language-server-protocol/>.
- [2] J. B. Almeida, J. B. Almeida, M. Barbosa и др., «Jasmin: High-Assurance and High-Speed Cryptography,» *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017. DOI: [10.1145/3133956.3134078](https://doi.org/10.1145/3133956.3134078).
- [3] G. Barthe, G. Barthe, F. Dupressoir и др., «EasyCrypt: A Tutorial,» 2013. DOI: [10.1007/978-3-319-10082-1_6](https://doi.org/10.1007/978-3-319-10082-1_6).
- [4] M. Dénès, *Jasmin language server*, вер. 0.0.1. url: <https://github.com/maximdenes/jasmin-language-server>.
- [5] Merlin development team, *Merlin*, вер. 5.4.1, 2025. url: <https://github.com/ocaml/merlin>.
- [6] F. Bour, T. Refis и G. Scherer, «Merlin: a language server for OCaml (experience report),» *Proc. ACM Program. Lang.*, т. 2, № ICFP, июль 2018. DOI: [10.1145/3236798](https://doi.org/10.1145/3236798).

- [7] Sengrep development team, *Sengrep*, вер. 1.122.0, 2025. url: <https://github.com/sengrep/sengrep/tree/develop/src/lsp>.
- [8] OCaml LSP development team, *OCaml Language Server Protocol implementation*, вер. 1.17.0. url: <https://ocaml.org/p/lsp/1.17.0>.
- [9] J. Vouillon и J. Dimino, *lwt, Promises and event-driven I/O*, вер. 5.9.1. url: <https://opam.ocaml.org/packages/lwt/>.
- [10] *JSON-RPC 2.0 Specification*, 2010. url: <https://www.jsonrpc.org/specification>.
- [11] OCaml community, *yojson*, вер. 2.2.2, 2024. url: <https://github.com/ocaml-community/yojson>.
- [12] M. Brunsfeld, A. Qureshi, A. Hlynskyi и др., *tree-sitter/tree-sitter: v0.25.5*, вер. v0.25.5, май 2025. DOI: [10.5281/zenodo.15531772](https://doi.org/10.5281/zenodo.15531772).
- [13] F. Pottier и Y. Régis-Gianas, *Menhir, parser generator for OCaml*. url: <https://gallium.inria.fr/~fpottier/menhir/>.